

240. 搜索二维矩阵 II

编写一个高效的算法来搜索 $m \times n$ 矩阵 `matrix` 中的一个目标值 `target`。该矩阵具有以下特性：

- 每行的元素从左到右升序排列。
- 每列的元素从上到下升序排列。

示例 1:

1	4	7	11	15
2	5	8	12	19
3	6	9	16	22
10	13	14	17	24
18	21	23	26	30

输入：matrix = [[1,4,7,11,15],[2,5,8,12,19],[3,6,9,16,22],[10,13,14,17,24],[18,21,23,26,30]], target = 5
输出：true

示例 2:

1	4	7	11	15
2	5	8	12	19
3	6	9	16	22
10	13	14	17	24
18	21	23	26	30

输入：matrix = [[1,4,7,11,15],[2,5,8,12,19],[3,6,9,16,22],[10,13,14,17,24],[18,21,23,26,30]], target = 20
输出：false

提示：

- `m == matrix.length`
- `n == matrix[i].length`
- `1 <= n, m <= 300`
- `109 <= matrix[i][j] <= 109`
- 每行的所有元素从左到右升序排列
- 每列的所有元素从上到下升序排列
- `109 <= target <= 109`

2. 解析

LeetCode 240: 搜索二维矩阵 II（详细解析）

1. 问题理解

目标： 在一个二维矩阵 `matrix` 中搜索目标值 `target`。

矩阵特性：

- 矩阵中的每一行从左到右递增。
- 矩阵中的每一列从上到下递增。

返回值： 如果矩阵中存在目标值，返回 `true`；否则，返回 `false`。

输入示例：

```
matrix = [  
  [1, 4, 7, 11, 15],  
  [2, 5, 8, 12, 19],  
  [3, 6, 9, 16, 22],  
  [10, 13, 14, 17, 24],  
  [18, 21, 23, 26, 30]  
]  
target = 5
```

输出示例： `true` (因为5在矩阵中)

2. 普通人的思路（及遇到的困难）

思路一：暴力遍历

想法： 最直观的方法是遍历整个矩阵，检查每个元素是否等于目标值。

实现： 使用两层循环，遍历矩阵的所有元素。

```
for (int i = 0; i < matrix.length; i++) {  
  for (int j = 0; j < matrix[0].length; j++) {  
    if (matrix[i][j] == target) {  
      return true;  
    }  
  }  
}
```

```
}  
return false;
```

缺点：

- 时间复杂度为 $O(m*n)$ ， m 和 n 分别是矩阵的行数和列数。
- 没有利用矩阵的特性（行和列的递增性质）。

思路二：对每一行进行二分查找

想法： 既然每一行是排序的，我们可以对每一行使用二分查找。

实现： 遍历每一行，对每一行使用二分查找查找目标值。

缺点：

- 时间复杂度为 $O(m*\log(n))$ ，比暴力法好，但仍然没有充分利用矩阵的所有特性。
- 只利用了行递增的特性，没有利用列递增的特性。

3. 更好的方法（从右上角或左下角开始搜索）

关键洞察： 我们可以从矩阵的右上角（或左下角）开始搜索，利用矩阵的行列都有序的特性，每次可以排除一行或一列。

思路：从右上角开始

从矩阵的右上角 `(0, n-1)` 开始：

- 如果当前元素等于目标值，返回 `true`。
- 如果当前元素大于目标值，说明当前列的所有元素都大于目标值（因为列是递增的），可以排除当前列，向左移动（`j--`）。
- 如果当前元素小于目标值，说明当前行的所有元素都小于目标值（因为行是递增的），可以排除当前行，向下移动（`i++`）。

为什么这样有效？

右上角元素有一个特殊性质：它是所在行的最大元素，所在列的最小元素。这让我们每次比较后都能排除一行或一列，缩小搜索范围。

4. 示例演示（使用右上角方法）

例如，搜索 `target = 5` 在上面示例矩阵中的过程：

1. 从右上角 `matrix[0][4] = 15` 开始：
 - `15 > 5`，排除第5列，向左移动，`j = 3`
2. 检查 `matrix[0][3] = 11`：
 - `11 > 5`，排除第4列，向左移动，`j = 2`
3. 检查 `matrix[0][2] = 7`：
 - `7 > 5`，排除第3列，向左移动，`j = 1`
4. 检查 `matrix[0][1] = 4`：
 - `4 < 5`，排除第1行，向下移动，`i = 1`
5. 检查 `matrix[1][1] = 5`：
 - `5 == 5`，找到目标值，返回 `true`

5. 代码实现

```
/*
 * @lc app=leetcode.cn id=240 lang=java
 * @lcpr version=30204
 *
 * [240] 搜索二维矩阵 II
 */

// @lcpr-template-start

// @lcpr-template-end
// @lc code=start
class Solution {
    /**
     * 在二维矩阵中搜索目标值
     *
     * @param matrix 行列分别递增的二维矩阵
     * @param target 目标值
     * @return 如果找到目标值返回true，否则返回false
     */
    public boolean searchMatrix(int[][] matrix, int target) {
```

```

// 检查矩阵是否为空
if (matrix == null || matrix.length == 0 || matrix[0].length == 0) {
    return false;
}

int rows = matrix.length;    // 矩阵的行数
int cols = matrix[0].length; // 矩阵的列数

// 从右上角开始搜索 (第0行, 最后一列)
int row = 0;                // 当前行索引, 初始为第一行 (上边)
int col = cols - 1;         // 当前列索引, 初始为最后一列 (右边)

// 当搜索位置在矩阵范围内时继续搜索
while (row < rows && col >= 0) {
    // 获取当前位置的元素
    int current = matrix[row][col];

    // 如果找到目标值, 返回true
    if (current == target) {
        return true;
    }
    // 如果当前值大于目标值, 目标必须在左侧
    // 因为当前列的所有值都大于或等于当前值, 排除这一列
    else if (current > target) {
        col--; // 向左移动 (减少列索引)
    }
    // 如果当前值小于目标值, 目标必须在下方
    // 因为当前行的所有值都小于或等于当前值, 排除这一行
    else { // current < target
        row++; // 向下移动 (增加行索引)
    }
}

// 搜索完毕仍未找到目标值
return false;
}
}
// @lc code=end

```

6. 代码解析

- **初始位置选择**：从右上角 `(0, cols-1)` 开始搜索，这是一个策略性选择，让我们能同时利用行和列的递增特性。
- **搜索循环**：每次比较当前元素和目标值，然后决定是向左还是向下移动。
- **三种情况**：
 - 当前元素等于目标值：返回 `true`。
 - 当前元素大于目标值：向左移动 (`col--`)，排除当前列。
 - 当前元素小于目标值：向下移动 (`row++`)，排除当前行。
- **边界检查**：当 `row` 超出行数或 `col` 变为负数时，表示已搜索完所有可能包含目标值的区域，退出循环。

7. 复杂度分析

时间复杂度： $O(m + n)$

- 最坏情况下，需要从右上角走到左下角，最多经过 $m + n - 1$ 个元素。
- 每一步都能排除一行或一列，所以时间复杂度是 $O(m + n)$ 。

空间复杂度： $O(1)$

- 只使用了几个额外变量 (`rows`, `cols`, `row`, `col`, `current`)。
- 这些变量的数量是常数级的，与矩阵大小无关。

8. 如何在面试中讲解

1. 确认问题理解：

- "这个问题是在行列有序的二维矩阵中搜索目标值。每行从左到右递增，每列从上到下递增。"

2. 讨论暴力法和二分法：

- "最直观的方法是遍历整个矩阵，时间复杂度为 $O(m*n)$ 。"
- "考虑到每行已排序，可以对每行使用二分查找，但这样时间复杂度仍为 $O(m*\log(n))$ 。"

3. 引入更优解法：

- "我们可以利用矩阵的行列都有序的特性，从右上角开始搜索。"

- "右上角元素有个特点：它是所在行的最大值，所在列的最小值。这让我们可以排除一行或一列。"

4. 解释搜索逻辑：

- "如果当前元素等于目标值，直接返回true。"
- "如果当前元素大于目标值，排除当前列，向左移动。"
- "如果当前元素小于目标值，排除当前行，向下移动。"

5. 分析复杂度：

- "时间复杂度为 $O(m + n)$ ，因为每一步都能排除一行或一列。"
- "空间复杂度为 $O(1)$ ，只需要常数级额外空间。"

6. 说明其他可行方案：

- "也可以从左下角开始搜索，逻辑类似但方向相反：大于目标值向上，小于目标值向右。"

通过这种方法，我们巧妙地利用了矩阵的特性，设计了一个线性时间复杂度的算法，远优于暴力搜索和每行二分查找的方法。

找到具有 2 个许可证类型的类似代码